

Lab no: 1

Date: 2026-06-19

Title: WAP to implement Caesar Cipher Algorithm.

Caesar Cipher:

Caesar Cipher is one of the oldest and simplest encryption algorithms. It is a type of monoalphabetic substitution cipher, meaning each letter in the plaintext is substituted with another letter. Each letter in the plaintext is replaced by a letter some fixed number of positions forward (for encryption) or backward (for decryption) in the alphabet. The shift value is known as the key.

Algorithm:

1. Start.
2. Input plaintext and key.
3. Convert each letter of the plaintext to its corresponding number (A = 0, B = 1, ..., Z = 25).
4. Encrypt each letter using: $C = (P + K) \bmod 26$, where P is the plaintext letter and K is the key.
5. Convert the resulting numbers back to letters to form the ciphertext.
6. To decrypt, use: $P = (C - K + 26) \bmod 26$, where C is the ciphertext letter.
7. Convert the resulting numbers back to letters to retrieve the original plain text.
8. Stop.

Source Code:

```
#include <stdio.h>
#include <ctype.h>
void caesar_encrypt(char *text,int shift,char
*result){ int i=0;
    while(text[i]){
        if(isalpha(text[i])){
            char base=isupper(text[i])?'A':'a';
            result[i]=base+(text[i]-base+shift)%26;
        }else{ result[i]=text[i];}
        i++;
    }
    result[i]='\0';
}
void caesar_decrypt(char *text,int shift,char
*result){
    int i=0;
    while(text[i]){
        if(isalpha(text[i])){
            char base=isupper(text[i])?'A':'a';
```

```

        result[i]=base+(text[i]-base-
shift+26)%26;
    }else{
        result[i]=text[i];
    }
    i++;
}
result[i]='\0';
}

int main(){
    char text[100],encrypted[100],decrypted[100];
    int shift;

    printf("Enter the text: ");
    scanf("%s",text);

    printf("Enter the shift value: ");
    scanf("%d",&shift);

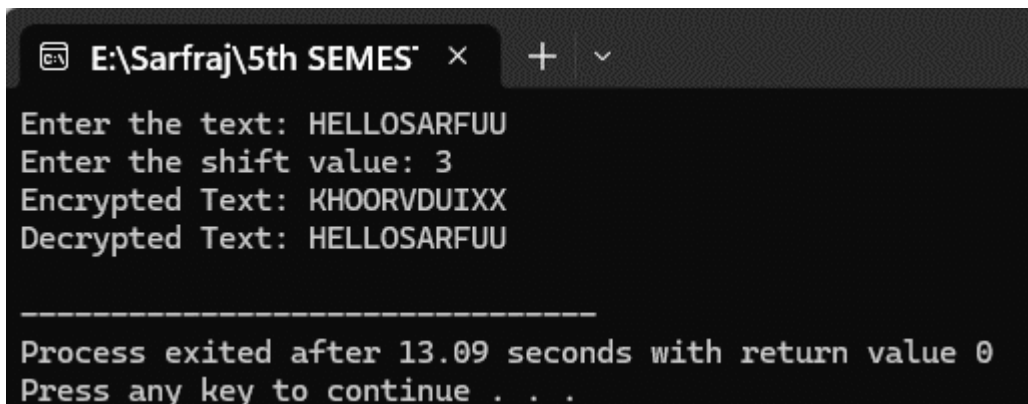
    caesar_encrypt(text,shift,encrypted);
    caesar_decrypt(encrypted,shift,decrypted);

    printf("Encrypted Text: %s\n",encrypted);
    printf("Decrypted Text: %s\n",decrypted);

    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter the text: HELLOSARFUU
Enter the shift value: 3
Encrypted Text: KHOORVDUIXX
Decrypted Text: HELLOSARFUU

-----
Process exited after 13.09 seconds with return value 0
Press any key to continue . . .

```

Title: WAP to implement Polyalphabetic Caesar Cipher Algorithm (Vignere's).

Vignère Cipher: The Vignère Cipher, often referred to as a polyalphabetic Caesar Cipher, is a polyalphabetic substitution cipher that improves upon the monoalphabetic Caesar Cipher by using a keyword to apply multiple shift values for encryption. Each letter of the plaintext is shifted by the numerical value of the corresponding letter in a repeating keyword (A=0, B=1, ..., Z=25), making it more secure against frequency analysis attacks. The key is extended to match the plaintext length by repeating the keyword, and non-alphabetic characters in the plaintext remain unchanged during encryption and decryption.

Algorithm

1. Start.
2. Input plaintext and a keyword (key).
3. Extend the key to match the plaintext length by repeating the keyword, ignoring non-alphabetic characters in the plaintext.
4. Convert each letter of the plaintext and the extended key to its corresponding number (A=0, B=1, ..., Z=25).
5. Encrypt each plaintext letter using: $C_i = (P_i + K_i) \bmod 26$, where P_i is the i-th plaintext letter's numerical value, K_i is the i-th key letter's numerical value, and C_i is the resulting ciphertext letter.
6. Convert the resulting numbers back to letters to form the ciphertext, preserving case and non-alphabetic characters.
7. To decrypt, use: $P_i = (C_i - K_i + 26) \bmod 26$, where C_i is the i-th ciphertext letter's numerical value.
8. Convert the resulting numbers back to letters to retrieve the original plaintext, preserving case and non-alphabetic characters.
9. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int char_to_int(char c){
    return tolower(c) - 'a';
}
void extend_key(const char *text,const char *key,char *ek){
    int klen=strlen(key),j=0;
    for(int i=0;text[i];i++){
        if(isalpha(text[i]))
            ek[i]=key[(j++)%klen];
        else
            ek[i]=text[i];
    }
    ek[strlen(text)]='\0';
```

```

}

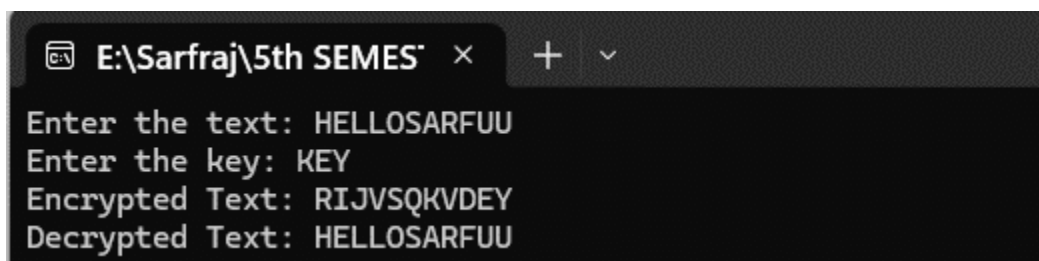
void vigenere_encrypt(const char *text,const char *key,char *res){
    char ek[100];
    extend_key(text,key,ek);
    for(int i=0;text[i];i++){
        if(isalpha(text[i])){
            char base=isupper(text[i])?'A':'a';
            res[i]=base+((text[i]-base)+char_to_int(ek[i]))%26;
        }else
            res[i]=text[i];
    }
    res[strlen(text)]='\0';
}

void vigenere_decrypt(const char *text,const char *key,char *res){
    char ek[100];
    extend_key(text,key,ek);
    for(int i=0;text[i];i++){
        if(isalpha(text[i])){
            char base=isupper(text[i])?'A':'a';
            res[i]=base+((text[i]-base)-char_to_int(ek[i])+26)%26;
        }else
            res[i]=text[i];
    }
    res[strlen(text)]='\0';
}

int main(){
    char text[100],key[100],enc[100],dec[100];
    printf("Enter the text: ");
    fgets(text,sizeof(text),stdin);
    text[strcspn(text,"\n")]='\0';
    printf("Enter the key: ");
    scanf("%s",key);
    vigenere_encrypt(text,key,enc);
    vigenere_decrypt(enc,key,dec);
    printf("Encrypted Text: %s\n",enc);
    printf("Decrypted Text: %s\n",dec);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter the text: HELLOSARFUU
Enter the key: KEY
Encrypted Text: RIJVSQKVDEY
Decrypted Text: HELLOSARFUU

```

Title: WAP to implement Playfair Cipher

Playfair Cipher : The Playfair Cipher is a digraph substitution cipher that encrypts pairs of letters (digraphs) using a 5x5 matrix constructed from a keyword. Unlike monoalphabetic ciphers like the Caesar Cipher, it processes two letters at a time, making it more secure against simple frequency analysis. The matrix is built by filling in the keyword (with duplicates removed and I/J treated as one letter) followed by the remaining alphabet letters (excluding J). During encryption, plaintext is divided into digraphs, with a filler letter 'X' added for identical letters in a pair or for odd-length text. Non-alphabetic characters remain unchanged. The encryption rules depend on the positions of the digraph letters in the matrix: same row, same column, or forming a rectangle. Decryption reverses these rules.

Algorithm

1. Start.
2. Input plaintext and keyword.
3. Generate a 5x5 matrix from the keyword (duplicates removed, I/J combined) and remaining alphabet (excluding J).
4. Form digraphs from plaintext: insert 'X' for identical letters or odd length, preserve non-alphabetic characters.
5. Encrypt each digraph using the matrix:
 - **Same row:** shift each letter right (wrap around).
 - **Same column:** shift each letter down (wrap around).
 - **Rectangle/ Diagonal:** swap each letter with the one in its row and the other's column.
6. Decrypt by reversing: shift left for same row, up for same column, same for rectangle.
7. Preserve letter case and non-alphabetic characters.
8. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void generate_matrix(const char *key,char m[5][5]){
    int used[26]={0},k=0,idx=0;
    char temp[26];
    for(int i=0;key[i];i++){
        char c=tolower(key[i]);
        if(c=='j') c='i';
        if(isalpha(c)&&!used[c-'a']){
            temp[k++]=c;
            used[c-'a']=1;
        }
    }
}
```

```

    for(char c='a';c<='z';c++){
        if(c=='j') continue;
        if(!used[c-'a']){
            temp[k++]=c;
            used[c-'a']=1;
        }
    }
    for(int i=0;i<5;i++)
        for(int j=0;j<5;j++)
            m[i][j]=temp[idx++];
}

void find_position(char c,char m[5][5],int *r,int *col){
    if(c=='j') c='i';
    c=tolower(c);
    for(int i=0;i<5;i++)
        for(int j=0;j<5;j++)
            if(m[i][j]==c){
                *r=i;
                *col=j;
                return;
            }
}

void process(const char *text,char m[5][5],int enc,char *res){
    char alpha[100]="",dig[100]="";
    int k=0,d=0;
    for(int i=0;text[i];i++)
        if(isalpha(text[i]))
            alpha[k++]=tolower(text[i]);
    for(int i=0;i<k;i++){
        dig[d++]=alpha[i];
        if(i+1<k && alpha[i]==alpha[i+1])
            dig[d++]='x';
        else if(i+1==k)
            dig[d++]='x';
        else
            dig[d++]=alpha[++i];
    }
    int len=0;
    for(int i=0;i<d;i+=2){
        int r1,c1,r2,c2;
        find_position(dig[i],m,&r1,&c1);
        find_position(dig[i+1],m,&r2,&c2);
        if(r1==r2){
            int s=enc?1:-1;

```

```

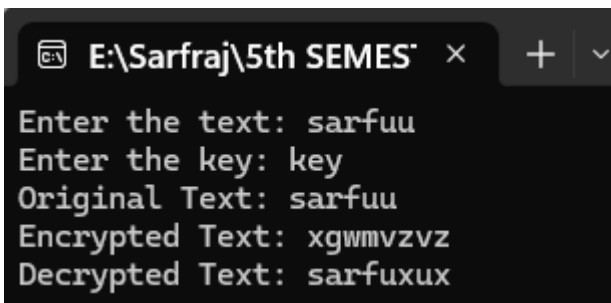
        res[len++]=m[r1][(c1+s+5)%5];
        res[len++]=m[r2][(c2+s+5)%5];
    }
    else if(c1==c2){
        int s=enc?1:-1;
        res[len++]=m[(r1+s+5)%5][c1];
        res[len++]=m[(r2+s+5)%5][c2];
    }
    else{
        res[len++]=m[r1][c2];
        res[len++]=m[r2][c1];
    }
}
res[len]='\0';
}

void playfair(const char *text,const char *key,int enc,char *out){
    char m[5][5];
    generate_matrix(key,m);
    process(text,m,enc,out);
}

int main(){
    char text[100],key[100],enc[100],dec[100];
    printf("Enter the text: ");
    fgets(text,sizeof(text),stdin);
    text[strlen(text)]='\0';
    printf("Enter the key: ");
    scanf("%s",key);
    playfair(text,key,1,enc);
    playfair(enc,key,0,dec);
    printf("Original Text: %s\n",text);
    printf("Encrypted Text: %s\n",enc);
    printf("Decrypted Text: %s\n",dec);
    return 0;
}

```

Output:



The screenshot shows a Windows command prompt window with the title bar "E:\Sarfraj\5th SEMES". The window contains the following text:

```

Enter the text: sarfuu
Enter the key: key
Original Text: sarfuu
Encrypted Text: xgwmvzvz
Decrypted Text: sarfuxux

```

Title: WAP to implement Hill Cipher

Hill Cipher

The Hill Cipher is a polygraphic substitution cipher that encrypts blocks of letters using linear algebra. It employs a key matrix (e.g., 2x2) to transform plaintext into ciphertext via matrix multiplication modulo 26. Each block of plaintext letters is converted to numerical values (A=0, B=1, ..., Z=25), multiplied by the key matrix, and converted back to letters. Decryption uses the inverse key matrix. The key matrix must be invertible modulo 26 (i.e., its determinant must be coprime with 26). Non-alphabetic characters remain unchanged, and padding (e.g., 'x') is added if the text length doesn't match the block size.

Algorithm

1. **Start**
2. **Input Plaintext and Key Matrix**
 - Provide plaintext message and a square invertible key matrix (e.g., 2x2 or 3x3).
3. **Convert Plaintext to Numerical Form**
 - Map letters to numbers: A=0, B=1, ..., Z=25.
4. **Divide Plaintext into Blocks**
 - Split plaintext into blocks of size n (dimension of the key matrix). □
Padding: If the last block is incomplete, add "X" to fill it.
5. **Encrypt Each Block**
 - Multiply each block (as a vector) by the key matrix: $C = K * P \bmod 26$
(C = ciphertext vector, K = key matrix, P = plaintext vector).
6. **Convert Numbers to Ciphertext**
 - Map resulting numerical vectors back to letters.
7. **Decrypt Using Inverse Key Matrix**
 - Compute the modular inverse of the key matrix ($K^{-1} \bmod 26$). □
Multiply ciphertext vectors by K^{-1} :
 $P = K^{-1} * C \bmod 26$.
8. **Convert Numbers to Plaintext**
 - Map decrypted numerical vectors back to letters (remove padding "X" if added).
9. **Stop**

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
int ctoi(char c){
```



```

        return tolower(c)-'a';}
char itoc(int n){
    return 'a'+((n%26+26)%26);
}
int gcd(int a,int b){
    while(b){
        int t=a%b;
        a=b;
        b=t;
    }
    return a;
}
int mod_inverse(int a,int m){
    for(int x=1;x<m;x++){
        if((a*x)%m==1)
            return x;
    }
    return -1;
}
int valid_key(int k[2][2]){
    int det=(k[0][0]*k[1][1]-k[0][1]*k[1][0])%26;
    det=(det+26)%26;
    return gcd(det,26)==1;
}
void inverse_key(int k[2][2],int inv[2][2]){
    int det=(k[0][0]*k[1][1]-k[0][1]*k[1][0])%26;
    det=(det+26)%26;
    int d=mod_inverse(det,26);
    inv[0][0]=(k[1][1]*d)%26;
    inv[0][1]=(-k[0][1]*d+26)%26;
    inv[1][0]=(-k[1][0]*d+26)%26;
    inv[1][1]=(k[0][0]*d)%26;
}
void hill(const char *text,int k[2][2],char *res){
    char a[100]="";
    int n=0;
    for(int i=0;text[i];i++){
        if(isalpha(text[i]))
            a[n++]=tolower(text[i]);
        if(n%2) a[n++]='x';
    }
    for(int i=0;i<n;i+=2){
        int p1=ctoi(a[i]);
        int p2=ctoi(a[i+1]);
        res[i]=itoc((k[0][0]*p1+k[0][1]*p2)%26);
        res[i+1]=itoc((k[1][0]*p1+k[1][1]*p2)%26);
    }
}

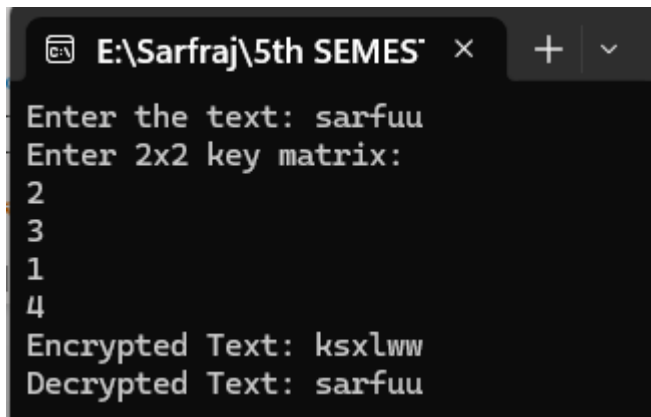
```

```

    }
    res[n]='\0';
}
int main(){
    char text[100],enc[100],dec[100];
    int key[2][2],inv[2][2];
    printf("Enter the text: ");
    fgets(text,sizeof(text),stdin);
    text[strcspn(text,"\n")]='\0';
    printf("Enter 2x2 key matrix:\n");
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            scanf("%d",&key[i][j]);
    if(!valid_key(key)){
        printf("Invalid key!\n");
        return 1;
    }
    hill(text,key,enc);
    inverse_key(key,inv);
    hill(enc,inv,dec);
    printf("Encrypted Text: %s\n",enc);
    printf("Decrypted Text: %s\n",dec);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES
Enter the text: sarfuu
Enter 2x2 key matrix:
2
3
1
4
Encrypted Text: ksxlww
Decrypted Text: sarfuu

```

Title: WAP to implement Rail fence Cipher

Rail Fence Cipher: The Rail Fence Cipher is a type of transposition cipher. It works by writing the plaintext in a zigzag pattern (like a rail fence) across multiple "rails" (rows) and then reading the letters row by row to get the ciphertext. It's simple but effective for basic obfuscation. The number of rails is the key, and encryption does not change the characters, only their order.

Algorithm:

The Rail Fence Cipher operates as follows:

1. Encryption:

- The plaintext is written in a zigzag pattern across a fixed number of rails (rows), determined by the key.
- The pattern alternates between moving down and up the rails, repeating every $2 * (\text{rails} - 1)$ positions.
- The ciphertext is formed by reading the characters row by row from the resulting grid.

2. Decryption:

- The ciphertext is used to reconstruct the zigzag pattern by determining the positions of characters in the grid based on the number of rails.
- The plaintext is recovered by reading the characters column by column, following the same zigzag path used during encryption.

Source Code:

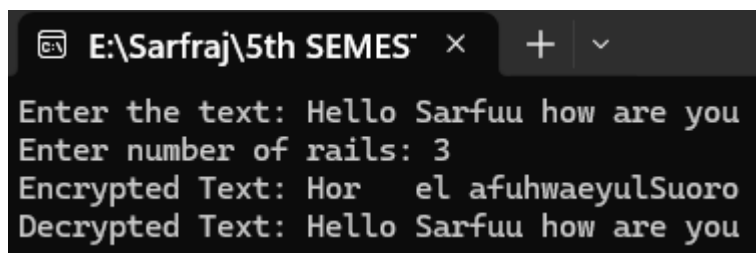
```
#include <stdio.h>
#include <string.h>
void encrypt(const char *text,int rails,char *enc){
    int len=strlen(text),idx=0,row=0,dir=1;
    char fence[rails][len];
    memset(fence,0,sizeof(fence));
    for(int i=0;i<len;i++){
        fence[row][i]=text[i];
        row+=dir;
        if(row==rails-1 || row==0)
            dir*=-1;
    }
    for(int r=0;r<rails;r++)
        for(int c=0;c<len;c++)
            if(fence[r][c])
                enc[idx++]=fence[r][c];
    enc[idx]='\0';
}
void decrypt(const char *enc,int rails,char *dec){
    int len=strlen(enc),idx=0,row=0,dir=1;
```

```

char fence[rails][len];
memset(fence,0,sizeof(fence));
for(int i=0;i<len;i++){
    fence[row][i]='*';
    row+=dir;
    if(row==rails-1 || row==0)
        dir*=-1;
}
for(int r=0;r<rails;r++)
    for(int c=0;c<len;c++)
        if(fence[r][c]=='*')
            fence[r][c]=enc[idx++];
row=0;
dir=1;
for(int i=0;i<len;i++){
    dec[i]=fence[row][i];
    row+=dir;
    if(row==rails-1 || row==0)
        dir*=-1;
}
dec[len]='\0';
}
int main(){
    char text[100],enc[100],dec[100];
    int rails;
    printf("Enter the text: ");
    fgets(text,sizeof(text),stdin);
    text[strcspn(text,"\n")]='\0';
    printf("Enter number of rails: ");
    scanf("%d",&rails);
    if(rails<2){
        printf("Invalid number of rails!\n");
        return 1;
    }
    encrypt(text,rails,enc);
    decrypt(enc,rails,dec);
    printf("Encrypted Text: %s\n",enc);
    printf("Decrypted Text: %s\n",dec);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter the text: Hello Sarfuu how are you
Enter number of rails: 3
Encrypted Text: Hor el afuhwaeyulSuoro
Decrypted Text: Hello Sarfuu how are you

```

Lab no: 6

Date: 2025-03-10

Title: WAP to implement Euclidean Algorithm

Euclidean Algorithm : The Euclidean Algorithm is an efficient method for computing the Greatest Common Divisor (GCD) of two integers, a and b. Attributed to the ancient Greek mathematician Euclid, this algorithm is based on the principle that the GCD of two numbers remains the same when the larger number is replaced by its difference with the smaller number. Specifically, it uses the property that $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$, where % denotes the modulo operation. The algorithm iteratively applies this rule until the remainder is zero, at which point the last non-zero remainder is the GCD. This method is widely used in number theory, particularly in applications like simplifying fractions, solving Diophantine equations, and cryptographic algorithms.

Algorithm

The Euclidean Algorithm operates as follows:

1. Input:

- Take two integers, a and b (positive or negative). ○ If the numbers are negative, convert them to positive, as GCD is always positive.

2. Base Case:

- If b equals 0, the GCD is a.

3. Iterative/Recursive Step:

- Replace a with b and b with $a \% b$.
- Repeat until b becomes 0.

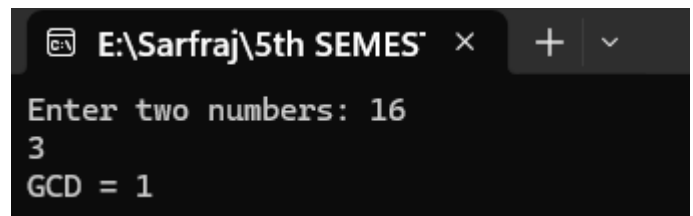
4. Output:

- The final non-zero value of a is the GCD of the original two numbers.

Source Code:

```
#include <stdio.h>
int gcd(int a,int b){
    while(b){
        int t=b;
        b=a%b;
        a=t;
    }
    return a;
}
int main(){
    int a,b;
    printf("Enter two numbers: ");
    scanf("%d%d",&a,&b);
    printf("GCD = %d\n",gcd(a,b));
    return 0;
}
```

Output:



```
E:\Sarfraj\5th SEMES x + v
Enter two numbers: 16
3
GCD = 1
```

Lab no: 7

Date: 2026-06-19

Title: WAP to implement Extended Euclidean Algorithm

Extended Euclidean Algorithm: The Extended Euclidean Algorithm is an extension of the Euclidean Algorithm that not only computes the Greatest Common Divisor (GCD) of two integers, a and b , but also finds integer coefficients x and y that satisfy Bézout's identity: $ax + by = \gcd(a, b)$. This algorithm, rooted in the work of Euclid and later formalized, is fundamental in number theory and has applications in cryptography, such as finding modular inverses for algorithms like RSA. It builds on the principle of the Euclidean Algorithm, $\gcd(a, b) = \gcd(b, a \% b)$, but tracks additional variables to derive the coefficients x and y . The algorithm ensures that the GCD is the largest positive integer that divides both a and b without a remainder, and the coefficients provide a linear combination equaling the GCD.

Algorithm

The Extended Euclidean Algorithm operates as follows:

1. **Input:**
 - Take two integers, a and b (positive or negative). ○ If negative, convert to positive for GCD calculation, adjusting coefficients as needed.
2. **Base Case:** ○ If b equals 0, the GCD is $|a|$, x is 1 (or -1 if a is negative), and y is 0.
3. **Recursive/Iterative Step:**
 - Compute $\gcd(b, a \% b)$ and coefficients x_1, y_1 for the recursive call.
 - Update coefficients: $x = y_1, y = x_1 - (a / b) * y_1$, where $/$ denotes integer division.
 - Repeat until b becomes 0.
4. **Output:**
 - Return the GCD and coefficients x, y satisfying $ax + by = \gcd(a, b)$.

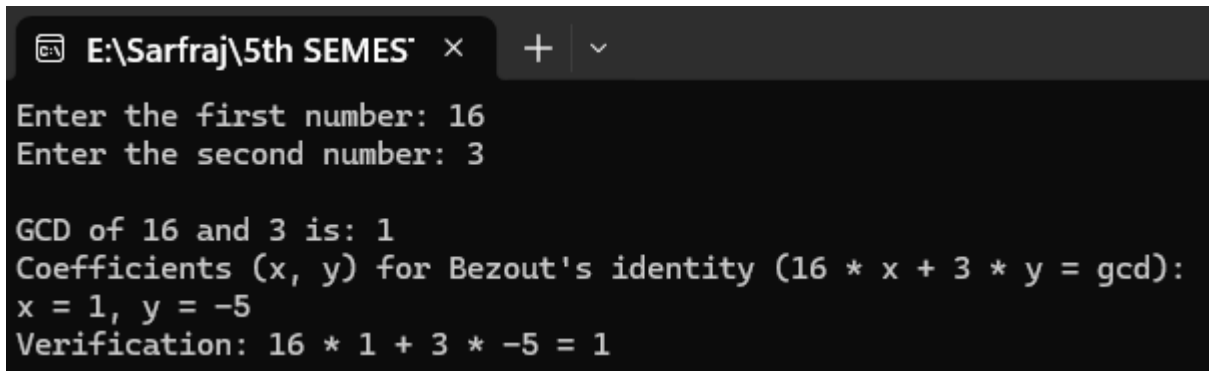
Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int extended_euclidean(int a, int b, int* x, int* y) {int original_a
= a; a=abs(a); b=abs(b);
    if (b == 0) {
        *x = (original_a < 0) ? -1 : 1;
        *y = 0;    return a;
    }    int x1, y1;    int gcd = extended_euclidean(b, a % b, &x1,
&y1);

    *x = y1;
    *y = x1 - (a / b) * y1;
    return gcd;
} int main() {    int num1, num2, x, y;
    printf("Enter the first number: ");    scanf("%d", &num1);
    printf("Enter the second number: ");    scanf("%d", &num2);
```

```
int gcd = extended_euclidean(num1, num2, &x, &y);  
printf("\nGCD of %d and %d is: %d\n", num1, num2, gcd);  
printf("Coefficients (x, y) for Bézout's identity (%d * x + %d * y  
= gcd):\n", num1, num2);    printf("x = %d, y = %d\n", x, y);  
printf("Verification: %d * %d + %d * %d = %d\n", num1, x,  
num2, y, num1 * x + num2 * y);  
return 0;  
}
```

Output:



```
E:\Sarfraj\5th SEMES × + ▾  
Enter the first number: 16  
Enter the second number: 3  
  
GCD of 16 and 3 is: 1  
Coefficients (x, y) for Bezout's identity (16 * x + 3 * y = gcd):  
x = 1, y = -5  
Verification: 16 * 1 + 3 * -5 = 1
```

Title: WAP to implement Miller-Rabin Primality Test

Miller-Rabin Primality Test

The Miller-Rabin Primality Test is a probabilistic algorithm used to determine whether a given number is prime or composite. Developed by Gary L. Miller in 1976 and refined by Michael O. Rabin in 1980, it leverages properties of modular arithmetic and Fermat's Little Theorem to efficiently test large numbers for primality. Unlike deterministic methods, the Miller-Rabin test is faster but provides a probabilistic result, with the accuracy increasing as the number of iterations (**k**) grows. The algorithm is based on the property that if n is prime, then for a randomly chosen base a (where $1 < a < n$), either $a^d \equiv 1 \pmod{n}$ or $a^{d \cdot 2^r} \equiv -1 \pmod{n}$ for some r , where $n - 1 = d \cdot 2^s$ and d is odd. If these conditions are not met, the number is composite. This test is widely used in cryptographic applications, such as RSA, due to its efficiency for large numbers.

Algorithm

The Miller-Rabin Primality Test operates as follows:

1. Input:

- Take a number n to test for primality and k , the number of iterations.
- Handle edge cases: if $n = 2$, it is prime; if $n < 2$ or n is even, it is composite.

2. Factorization:

- Express $n - 1$ as $d \cdot 2^s$, where d is odd and s is the number of times 2 divides $n - 1$.

3. Testing Loop:

- For k iterations:
 - Select a random base a in the range $[2, n-2]$.
 - Compute $x = a^d \pmod{n}$ using modular exponentiation.
 - If $x = 1$ or $x = n - 1$, continue to the next iteration.
 - For r from 0 to $s-1$, compute $x = x^2 \pmod{n}$. If $x = n - 1$, continue to the next iteration.
 - If none of these conditions are met, n is composite.

4. Output:

- If all iterations pass, n is likely prime.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
long long mod_pow(long long base, long long exp, long long mod){
    long long result=1;
    base%=mod;
    while(exp){
        if(exp&1)
```

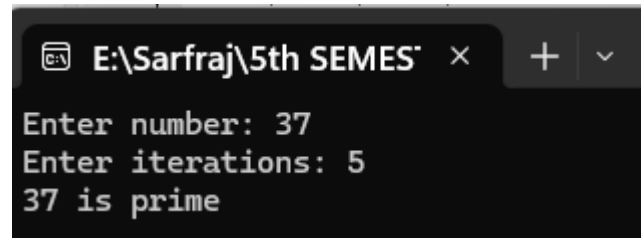


```

        result=(result*base)%mod;
        base=(base*base)%mod;
        exp/=2;
    }
    return result;
}
int miller_rabin(long long n,int k){
    if(n==2) return 1;
    if(n<2 || n%2==0) return 0;
    long long d=n-1;
    int s=0;
    while(d%2==0){
        d/=2;
        s++;
    }
    for(int i=0;i<k;i++){
        long long a=2+rand()%(n-3);
        long long x=mod_pow(a,d,n);
        if(x==1 || x==n-1)
            continue;
        int flag=0;
        for(int r=0;r<s-1;r++){
            x=(x*x)%n;
            if(x==n-1){
                flag=1;
                break;
            }
        }
        if(!flag)
            return 0;
    }
    return 1;
}
int main(){
    long long n;
    int k;
    printf("Enter number: ");    scanf("%lld",&n);
    printf("Enter iterations: ");    scanf("%d",&k);
    srand(time(NULL));
    printf("%lld is %s\n",n,miller_rabin(n,k) ? "prime" : "composite");
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter number: 37
Enter iterations: 5
37 is prime

```

Title: WAP to implement Euler's Theorem

Euler's Theorem : Euler's Theorem is a fundamental result in number theory, named after Leonhard Euler, which states that if two integers a and n are coprime (i.e., their greatest common divisor is 1), then a raised to the power of Euler's totient function of n , denoted $\phi(n)$, is congruent to 1 modulo n : $a^{\phi(n)} \equiv 1 \pmod{n}$. The totient function $\phi(n)$ counts the number of integers from 1 to n that are coprime with n . For a prime number p , $\phi(p) = p - 1$, and for a product of distinct primes, $\phi(n)$ is computed as n times the product of $(1 - 1/p_i)$ for each prime factor p_i of n . This theorem is a generalization of Fermat's Little Theorem and is crucial in cryptographic algorithms like RSA, where it underpins modular exponentiation and the computation of modular inverses.

Algorithm

Euler's Theorem is demonstrated as follows:

1. **Input:** ○ Take two integers: a (the base) and n (the modulus, positive).
2. **Validation:**
 - Check if $n > 0$; if not, the modulus is invalid.
 - Check if $a > 0$; if not, the base is invalid.
 - Compute $\text{GCD}(a, n)$ using the Euclidean Algorithm; if $\text{GCD} \neq 1$, a and n are not coprime, and the theorem does not apply.
3. **Compute Euler's Totient Function ($\phi(n)$):**
 - Initialize $\text{result} = n$.
 - For each prime factor p of n (found by checking divisors up to $\sqrt{n}$):
 - If n is divisible by p , divide n by p repeatedly and update result as $\text{result} - \text{result}/p$.
 - If $n > 1$ after the loop, account for the remaining prime factor.
4. **Modular Exponentiation:**
 - Compute $a^{\phi(n)} \bmod n$ using an efficient modular exponentiation algorithm.
5. **Output:**
 - If a and n are coprime, the result should be 1, verifying Euler's Theorem.
 - Handle error cases (invalid modulus, base, or non-coprime inputs).

Source Code:

```
#include <stdio.h>
int gcd(int a,int b){
    while(b){
        int t=b;
        b=a%b;
        a=t;
    }
    return a;}

```

```

int phi(int n){ int result=n;
    for(int i=2;i*i<=n;i++){
        if(n%i==0){
            while(n%i==0)
                n/=i;
            result-=result/i;}}
    if(n>1)
        result-=result/n;
    return result;}

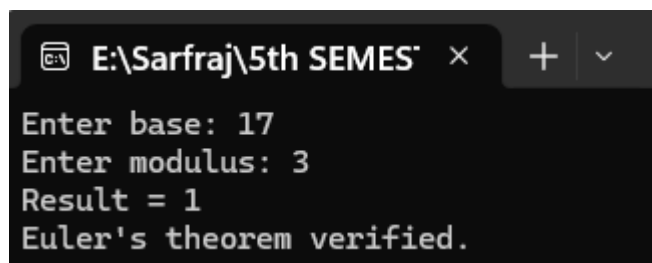
long long mod_pow(long long base,long long
exp,long long mod){
    long long result=1;
    base%=mod;
    while(exp){
        if(exp&1)
            result=(result*base)%mod;
        base=(base*base)%mod;
        exp/=2; }
    return result;}

long long euler_theorem(int a,int n){
    if(n<=0) return -1; if(a<=0) return -2;
    if(gcd(a,n)!=1) return -3;
    return mod_pow(a,phi(n),n); }

int main(){
    int a,n;
    printf("Enter base: ");    scanf("%d",&a);
    printf("Enter modulus: "); scanf("%d",&n);
    long long ans=euler_theorem(a,n);
    if(ans==-1) printf("Invalid modulus\n");
    else if(ans==-2)    printf("Invalid base\n");
    else if(ans==-3)    printf("a and n must be
coprime\n");
    else{ printf("Result = %lld\n",ans);
        printf("Euler's theorem verified.\n");}
    return 0; }

```

Output:



```

E:\Sarfraj\5th SEMES >
Enter base: 17
Enter modulus: 3
Result = 1
Euler's theorem verified.

```

Title: WAP to find the prime roots of a number

Primitive Roots : In number theory, a primitive root modulo a prime number p is an integer a (where $1 \leq a < p$) such that the powers $a^1, a^2, \dots, a^{(p-1)}$ modulo p generate all integers from 1 to $p-1$ in some order. This implies that the order of a modulo p , defined as the smallest positive integer k such that $a^k \equiv 1 \pmod{p}$, must equal $p-1$. Primitive roots exist only for certain moduli, notably prime numbers, and are significant in cryptographic systems like Diffie-Hellman key exchange and discrete logarithm problems. For a prime p , the number of primitive roots is given by Euler's totient function $\phi(p-1)$, and they are found by testing whether $a^i \bmod p \neq 1$ for all i from 1 to $p-2$ and verifying that $a^{(p-1)} \bmod p = 1$, per Fermat's Little Theorem.

Algorithm

The process to find primitive roots of a prime number p is as follows:

1. Input: ○ Take a positive integer p and verify it is prime.
2. Validation:
 - If $p \leq 1$, it is invalid.
 - If p is not prime, primitive roots do not exist (except for $p = 2$, where 1 is the primitive root).
 - If $p = 2$, output 1 as the only primitive root.
3. Primitive Root Check:
 - For each integer a from 1 to $p-1$:
 - Compute $a^i \bmod p$ for i from 1 to $p-2$ using modular exponentiation.
 - If any $a^i \bmod p = 1$, a is not a primitive root.
 - If a passes the check, verify $a^{(p-1)} \bmod p = 1$. ▪ If both conditions hold, a is a primitive root.
4. Output:
 - List all primitive roots found or indicate if none exist (unlikely for a prime).

Source Code:

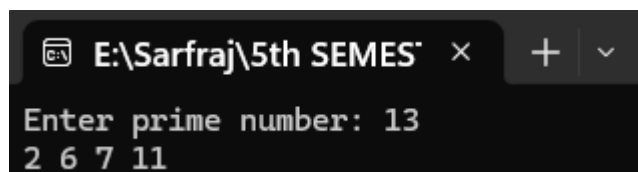
```
#include <stdio.h>
long long mod_pow(long long b, long long e, long long m){
    long long r=1;
    b%=m;
    while(e){
        if(e&1) r=(r*b)%m;
        b=(b*b)%m;
        e/=2;
    }
    return r;
}
int gcd(int a, int b){
    while(b){
```

```

        int t=b;
        b=a%b;
        a=t;
    }
    return a;
}
int is_prime(long long n){
    if(n==2) return 1;
    if(n<2 || n%2==0) return 0;
    for(long long i=3;i*i<=n;i+=2)
        if(n%i==0) return 0;
    return 1;
}
void primitive_roots(long long p){
    if(!is_prime(p)){
        printf("Input must be prime\n");
        return;
    }
    if(p==2){
        printf("1\n");
        return;
    }
    for(long long a=1;a<p;a++){
        int ok=1;
        for(long long i=1;i<p-1;i++){
            if(mod_pow(a,i,p)==1){
                ok=0;
                break;
            }
        }
        if(ok && mod_pow(a,p-1,p)==1)
            printf("%lld ",a);
    }
    printf("\n");
}
int main(){
    long long p;
    printf("Enter prime number:");    scanf("%lld",&p);
    primitive_roots(p);
    return 0;
}

```

Output:



The screenshot shows a Windows command prompt window with the title bar "E:\Sarfraj\5th SEMES". The prompt displays the input "Enter prime number: 13" and the output "2 6 7 11".

Title: WAP to compute the discrete logarithm of a number

Discrete Logarithm: The discrete logarithm problem is a cornerstone of number theory and cryptography, particularly in systems like Diffie-Hellman key exchange and ElGamal encryption. Given a prime number p , a primitive root a modulo p , and an integer b (where $1 \leq b < p$), the discrete logarithm is the unique exponent i such that $b \equiv a^i \pmod{p}$, where $0 \leq i < p$. This exponent is denoted as $\text{dlog}_{\{a,p\}}(b) = i$. A primitive root a of p generates all integers from 1 to $p-1$ through its powers $a^1, a^2, \dots, a^{(p-1)} \pmod{p}$, ensuring a unique solution for i . The difficulty of computing discrete logarithms for large primes underpins the security of many cryptographic algorithms, as no efficient general solution exists for large moduli.

Algorithm

The computation of the discrete logarithm operates as follows:

1. Input:

- Take three integers: a (the base, a primitive root), b (the target number), and p (a prime modulus).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Ensure a and b are positive and $b \not\equiv 0 \pmod{p}$.
- Confirm a is a primitive root of p by checking that $a^i \pmod{p} \neq 1$ for i from 1 to $p-2$ and $a^{(p-1)} \pmod{p} = 1$.

3. Computation:

- Use a brute-force approach: for i from 0 to $p-1$, compute $a^i \pmod{p}$ using modular exponentiation.
- If $a^i \pmod{p} = b \pmod{p}$, then i is the discrete logarithm.

4. Output:

- Return the discrete logarithm i if found, or indicate invalid inputs (nonprime p , non-primitive root a , or no solution).

Source Code:

```
#include <iostream>
using namespace std;
int gcd(int a,int b){
    while(b){int t=b;
        b=a%b;
        a=t;}
    return a;}
long long mod_pow(long long b,long long e,long long m){
    long long r=1;
    b%=m;
    while(e){
```

```

        if(e&1) r=(r*b)%m;
        b=(b*b)%m;
        e/=2;}
    return r;}

bool is_prime(long long n){
    if(n==2) return true;
    if(n<2 || n%2==0) return false;
    for(long long i=3;i*i<=n;i+=2)
        if(n%i==0) return false;
    return true;}

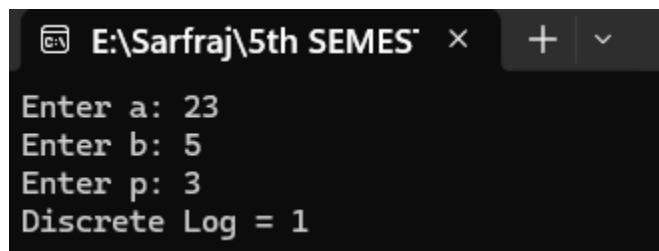
bool primitive_root(long long a,long long p){
    if(!is_prime(p) || gcd(a,p)!=1)    return false;
    for(long long i=1;i<p-1;i++)
        if(mod_pow(a,i,p)==1)    return false;
    return mod_pow(a,p-1,p)==1; }

long long discrete_log(long long a,long long b,long long p){
    if(!is_prime(p) || a<=0 || b<=0)    return -1;
    if(!primitive_root(a,p))    return -2;
    for(long long i=0;i<p;i++)
        if(mod_pow(a,i,p)==b%p)    return i;
    return -1;}

int main(){
    long long a,b,p;
    cout<<"Enter a: "; cin>>a;
    cout<<"Enter b: "; cin>>b;
    cout<<"Enter p: "; cin>>p;
    long long ans=discrete_log(a,b,p);
    if(ans==-1) cout<<"Invalid input\n";
    else if(ans==-2)    cout<<"a is not primitive root\n";
    else    cout<<"Discrete Log = "<<ans<<endl;
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter a: 23
Enter b: 5
Enter p: 3
Discrete Log = 1

```

Title: WAP to implement Diffie-Hellman Key Exchange

Diffie-Hellman Key Exchange : The Diffie-Hellman Key Exchange, introduced by Whitfield Diffie and Martin Hellman in 1976, is a cryptographic protocol that enables two parties, typically referred to as Alice and Bob, to establish a shared secret key over an insecure communication channel. This shared key can then be used for symmetric encryption. The security of the protocol relies on the computational difficulty of the discrete logarithm problem. It uses a publicly agreed-upon prime number p and a primitive root g modulo p . Each party selects a private key (a for Alice, b for Bob) and computes a public key ($g^a \bmod p$ and $g^b \bmod p$, respectively). By exchanging public keys and using their private keys, both parties compute the same shared secret, which is $g^{(a*b)} \bmod p$. The protocol's strength lies in the fact that computing the discrete logarithm to derive the private keys from the public keys is infeasible for large p .

Algorithm

The Diffie-Hellman Key Exchange operates as follows:

1. Input:

- Select a prime number p and a primitive root g modulo p .
- Alice chooses a private key a , and Bob chooses a private key b (both positive integers).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Confirm g is a primitive root of p by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure private keys a and b are positive.

3. Public Key Computation:

- Alice computes her public key: $A = g^a \bmod p$.
- Bob computes his public key: $B = g^b \bmod p$.

4. Key Exchange:

- Alice and Bob exchange their public keys A and B .

5. Shared Secret Computation:

- Alice computes the shared secret: $S = B^a \bmod p$.
- Bob computes the shared secret: $S = A^b \bmod p$.
- Since $B^a \bmod p = (g^b)^a \bmod p = g^{(a*b)} \bmod p = (g^a)^b \bmod p = A^b \bmod p$, both compute the same secret.

6. Output:

- Output the shared secret if valid, or indicate errors (non-prime p , invalid g , or negative private keys).

Source Code:

```
#include <stdio.h>
int gcd(int a,int b){
    while(b){int t=b;
        b=a%b; a=t;}
}
```

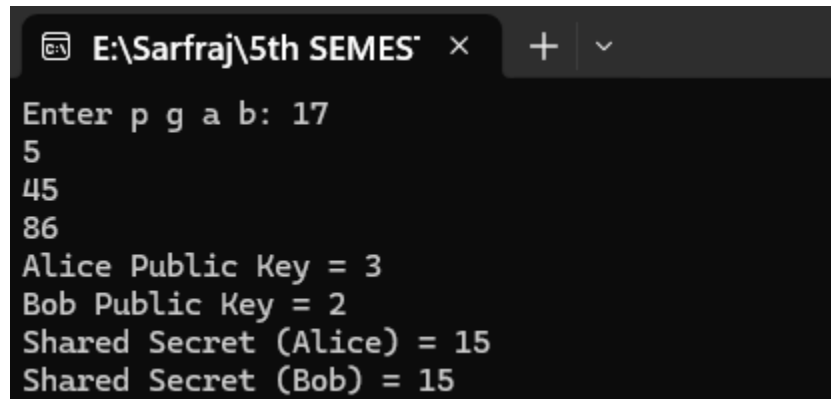


```

    return a;}
long long mod_pow(long long b,long long e,long long m){
    long long r=1;    b%=m;
    while(e){
        if(e&1) r=(r*b)%m;
        b=(b*b)%m;    e/=2;}
    return r;}
int is_prime(long long n){
    if(n==2) return 1;
    if(n<2 || n%2==0) return 0;
    for(long long i=3;i*i<=n;i+=2)
        if(n%i==0) return 0;
    return 1;}
int primitive_root(long long g,long long p){
    if(!is_prime(p) || gcd(g,p)!=1)    return 0;
    for(long long i=1;i<p-1;i++)
        if(mod_pow(g,i,p)==1)    return 0;
    return mod_pow(g,p-1,p)==1;}
void diffie(long long p,long long g,long long a,long long b){
    if(!is_prime(p)){printf("Invalid p\n");
        return;}
    if(!primitive_root(g,p)){
        printf("Invalid g\n");
        return;}
    long long A=mod_pow(g,a,p);    long long B=mod_pow(g,b,p);
    long long SA=mod_pow(B,a,p);    long long SB=mod_pow(A,b,p);
    printf("Alice Public Key = %lld\n",A);    printf("Bob Public Key = %lld\n",B);
    printf("Shared Secret (Alice) = %lld\n",SA);    printf("Shared Secret (Bob) = %lld\n",SB);
}
int main(){
    long long p,g,a,b;
    printf("Enter p g a b: ");    scanf("%lld%lld%lld%lld",&p,&g,&a,&b);
    diffie(p,g,a,b);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter p g a b: 17
5
45
86
Alice Public Key = 3
Bob Public Key = 2
Shared Secret (Alice) = 15
Shared Secret (Bob) = 15

```

Man-in-the-Middle Attack on Diffie-Hellman Key Exchange : The Man-in-the-Middle (MitM) attack is a significant vulnerability in the basic Diffie-Hellman Key Exchange protocol, which allows two parties, Alice and Bob, to establish a shared secret key over an insecure channel. In an MitM attack, an adversary, often referred to as Eve, intercepts the public keys exchanged between Alice and Bob and substitutes them with her own. This enables Eve to establish separate shared secrets with each party, effectively masquerading as Bob to Alice and as Alice to Bob. As a result, Eve can decrypt, read, modify, and re-encrypt messages, compromising communication while Alice and Bob believe they are communicating securely. The attack exploits the lack of authentication in the basic Diffie-Hellman protocol, as it does not verify the identity of the parties exchanging public keys. The security of the protocol relies on the difficulty of the discrete logarithm problem, but this does not protect against active interception.

Algorithm

The Man-in-the-Middle attack on the Diffie-Hellman Key Exchange operates as follows:

1. Setup:

- Alice and Bob agree on public parameters: a prime number p and a primitive root g modulo p .
- Alice selects a private key a , and Bob selects a private key b .
- Eve, the attacker, selects two private keys, $e1$ (for communication with Bob) and $e2$ (for communication with Alice).

2. Public Key Exchange and Interception:

- Alice computes her public key $A = g^a \bmod p$ and sends it to Bob, but Eve intercepts A and sends her own public key $E2 = g^{e2} \bmod p$ to Alice.
- Bob computes his public key $B = g^b \bmod p$ and sends it to Alice, but Eve intercepts B and sends her own public key $E1 = g^{e1} \bmod p$ to Bob.

3. Shared Secret Computation:

- Alice computes a shared secret with Eve: $S1 = E2^a \bmod p = g^{(e2*a)} \bmod p$.
- Bob computes a shared secret with Eve: $S2 = E1^b \bmod p = g^{(e1*b)} \bmod p$.
- Eve computes her shared secret with Alice: $S_{\text{eve_alice}} = A^{e2} \bmod p = g^{(a*e2)} \bmod p$ (matches $S1$).
- Eve computes her shared secret with Bob: $S_{\text{eve_bob}} = B^{e1} \bmod p = g^{(b*e1)} \bmod p$ (matches $S2$).

4. Communication:

- Alice encrypts messages using $S1$, which Eve can decrypt using $S_{\text{eve_alice}}$, modify if desired, and re-encrypt using $S_{\text{eve_bob}}$ to send to

Bob. ○ Similarly, Bob encrypts messages using S2, which Eve can decrypt and manipulate.

5. Output:

- Display the public keys, private keys, and shared secrets to demonstrate that Eve has successfully established separate shared secrets with Alice and Bob.

Source Code:

```
#include <stdio.h>
int gcd(int a,int b){
    while(b){
        int t=b;
        b=a%b;
        a=t;
    }
    return a;
}
long long mod_pow(long long b,long long e,long
long m){
    long long r=1;
    b%=m;
    while(e){
        if(e&1) r=(r*b)%m;
        b=(b*b)%m;
        e/=2;
    }
    return r;
}
int is_prime(long long n){
    if(n==2) return 1;
    if(n<2 || n%2==0) return 0;

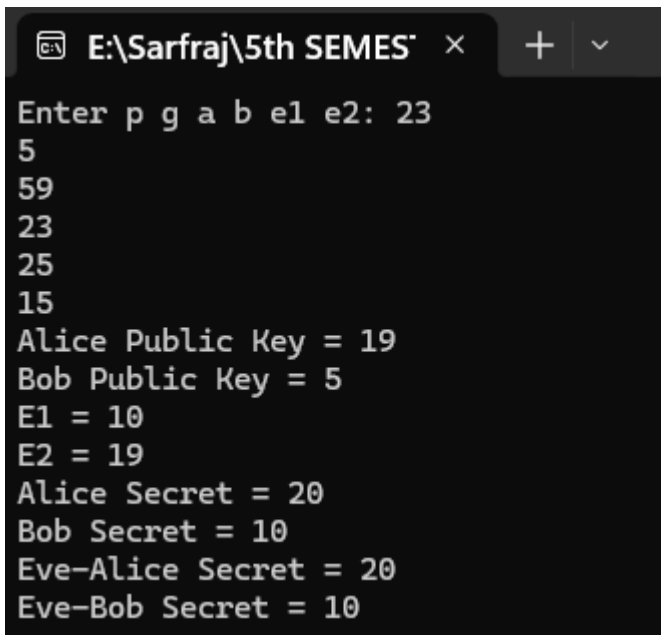
    for(long long i=3;i*i<=n;i+=2)
        if(n%i==0) return 0;
    return 1;
}
int primitive_root(long long g,long long p){
    if(!is_prime(p) || gcd(g,p)!=1)    return 0;
    for(long long i=1;i<p-1;i++)
        if(mod_pow(g,i,p)==1)    return 0;
    return mod_pow(g,p-1,p)==1; }
```

```

void mitm(long long p,long long g,long long
a,long long b,long long e1,long long e2){
    if(!is_prime(p) || !primitive_root(g,p)    return;
    long long A=mod_pow(g,a,p);
    long long B=mod_pow(g,b,p);
    long long E1=mod_pow(g,e1,p);
    long long E2=mod_pow(g,e2,p);
    long long S1=mod_pow(E2,a,p);
    long long S2=mod_pow(E1,b,p);
    long long SEA=mod_pow(A,e2,p);
    long long SEB=mod_pow(B,e1,p);
    printf("Alice Public Key = %lld\n",A);
    printf("Bob Public Key = %lld\n",B);
    printf("E1 = %lld\n",E1);
    printf("E2 = %lld\n",E2);
    printf("Alice Secret = %lld\n",S1);
    printf("Bob Secret = %lld\n",S2);
    printf("Eve-Alice Secret = %lld\n",SEA);
    printf("Eve-Bob Secret = %lld\n",SEB);
}
int main(){
    long long p,g,a,b,e1,e2;
    printf("Enter p g a b e1 e2: ");
    scanf("%lld%lld%lld%lld%lld%lld",&p,&g,&a,
&b,&e1,&e2);
    mitm(p,g,a,b,e1,e2);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
Enter p g a b e1 e2: 23
5
59
23
25
15
Alice Public Key = 19
Bob Public Key = 5
E1 = 10
E2 = 19
Alice Secret = 20
Bob Secret = 10
Eve-Alice Secret = 20
Eve-Bob Secret = 10

```

Title: WAP to implement RSA Algorithm

RSA Algorithm : The RSA (Rivest-Shamir-Adleman) algorithm, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman, is an asymmetric cryptographic system used for secure data transmission. It employs a public key (e, n) for encryption and a private key (d, n) for decryption, relying on the computational difficulty of factoring the product of two large prime numbers. RSA is widely used in secure communication protocols, such as HTTPS and digital signatures. The algorithm generates keys by selecting two large primes p and q , computing the modulus $n = p * q$ and the totient $\phi(n) = (p-1)(q-1)$, choosing a public exponent e coprime with $\phi(n)$, and calculating the private exponent d such that $d * e \equiv 1 \pmod{\phi(n)}$. Encryption transforms a message m into ciphertext $c = m^e \bmod n$, and decryption recovers $m = c^d \bmod n$. The security stems from the impracticality of factoring n for large primes.

Algorithm

The RSA algorithm operates as follows:

1. Key Generation:

- Select two large prime numbers p and q .
- Compute the modulus $n = p * q$.
- Compute Euler's totient function $\phi(n) = (p-1)(q-1)$.
- Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
- Compute the private exponent d using the modular inverse: $d * e \equiv 1 \pmod{\phi(n)}$.

2. Encryption:

- Take a message m (where $0 \leq m < n$).
- Compute the ciphertext $c = m^e \bmod n$ using modular exponentiation.

3. Decryption:

- Compute the original message $m = c^d \bmod n$ using modular exponentiation.

4. Validation:

- Verify p and q are prime using a primality test (e.g., trial division up to $\sqrt{n}$).
- Ensure e is coprime with $\phi(n)$ using the Euclidean algorithm.
- Check that m is in the range $[0, n)$.
- Confirm d exists by computing the modular inverse of e modulo $\phi(n)$.

5. Output:

- Display the public key (e, n) , private key (d, n) , ciphertext c , and decrypted message m .
- Verify decryption success if the decrypted message matches the original.

Source Code:

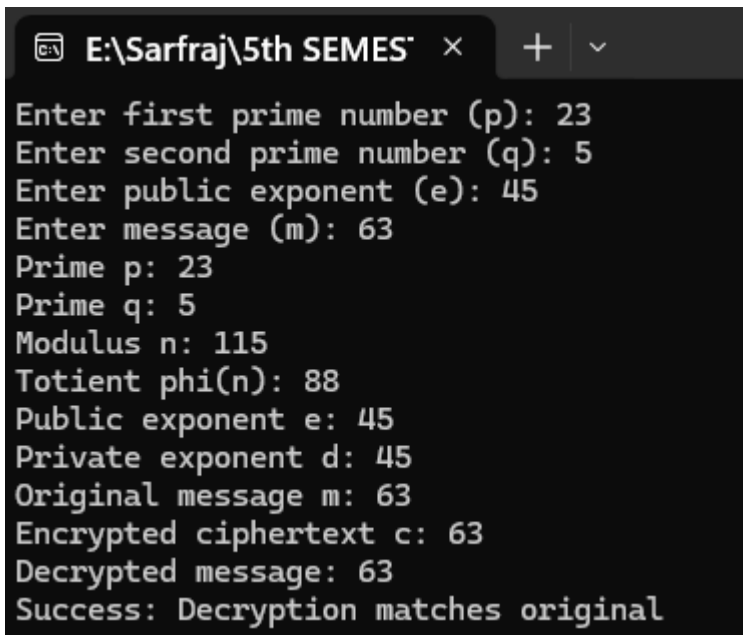
```
#include <stdio.h>
#include <stdbool.h>
long long gcd(long long a, long long b){
    while(b){    long long t=b;
        b=a%b;   a=t; }
    return a;
}
bool is_prime(long long n){
    if(n<=1) return false;
    if(n==2) return true;
    if(n%2==0) return false;
    for(long long i=3; i*i<=n; i+=2)
        if(n%i==0) return false;
    return true;
}
long long mod_pow(long long base, long long exp, long long mod){
    long long result=1;
    base%=mod;
    while(exp){
        if(exp&1)            result=(result*base)%mod;
        base=(base*base)%mod;    exp/=2; }
    return result; }
long long mod_inverse(long long e, long long phi){
    long long t=0, new_t=1;
    long long r=phi, new_r=e;
    while(new_r){ long long q=r/new_r;
        long long temp=t;        t=new_t;
        new_t=temp-q*new_t;    temp=r;
        r=new_r;                new_r=temp-q*new_r; }
    if(r>1) return -1;
    if(t<0) t+=phi;    return t; }
void rsa(long long p, long long q, long long e, long long m){
    if(!is_prime(p)||!is_prime(q)){ printf("Invalid: p and q must be prime\n");
        return; }
    long long n=p*q;
    long long phi=(p-1)*(q-1);
    if(e<=1||e>=phi){
        printf("Invalid: e must satisfy 1 < e < phi(n)\n");
        return; }
    if(gcd(e, phi)!=1){
        printf("Invalid: e must be coprime with phi(n)\n");
        return; }
```

```

if(m<0||m>=n){
    printf("Invalid: message m must be in range [0,n)\n");    return; }
long long d=mod_inverse(e,phi);
if(d==-1){
    printf("Invalid: modular inverse for e does not exist\n");
    return;
}
long long c=mod_pow(m,e,n);    long long dec=mod_pow(c,d,n);
printf("Prime p: %lld\n",p);    printf("Prime q: %lld\n",q);
printf("Modulus n: %lld\n",n);    printf("Totient phi(n): %lld\n",phi);
printf("Public exponent e: %lld\n",e);    printf("Private exponent d: %lld\n",d);
printf("Original message m: %lld\n",m);    printf("Encrypted ciphertext c: %lld\n",c);
printf("Decrypted message: %lld\n",dec);
if(m==dec)
    printf("Success: Decrypted message matches original\n");
else
    printf("Error: Decryption failed\n");
}
int main(){
    long long p,q,e,m;
    printf("Enter first prime number (p): ");    scanf("%lld",&p);
    printf("Enter second prime number (q): ");    scanf("%lld",&q);
    printf("Enter public exponent (e): ");    scanf("%lld",&e);
    printf("Enter message to encrypt (m): ");    scanf("%lld",&m);
    rsa(p,q,e,m);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES' x + v
Enter first prime number (p): 23
Enter second prime number (q): 5
Enter public exponent (e): 45
Enter message (m): 63
Prime p: 23
Prime q: 5
Modulus n: 115
Totient phi(n): 88
Public exponent e: 45
Private exponent d: 45
Original message m: 63
Encrypted ciphertext c: 63
Decrypted message: 63
Success: Decryption matches original

```

Title: WAP to implement ElGamal Cryptographic System Algorithm

ElGamal Cryptographic System : The ElGamal cryptographic system, developed by Taher ElGamal in 1985, is an asymmetric encryption algorithm based on the Diffie-Hellman key exchange. It uses a public key (p, g, h) for encryption and a private key a for decryption, with security rooted in the computational difficulty of the discrete logarithm problem. A large prime p and a primitive root g modulo p are chosen, and the public key h is computed as $g^a \bmod p$, where a is the private key. Encryption of a message m produces a ciphertext pair $(c1, c2)$, and decryption recovers the original message. ElGamal is used in applications like secure key exchange and digital signatures, offering probabilistic encryption due to the random integer k , which enhances security by producing different ciphertexts for the same message.

Algorithm

The ElGamal cryptographic system operates as follows:

1. Key Generation:

- Select a large prime p and a primitive root g modulo p .
- Choose a private key a ($1 \leq a \leq p-2$).
- Compute the public key $h = g^a \bmod p$ using modular exponentiation.

2. Encryption:

- Take a message m ($0 \leq m < p$) and a random integer k ($1 \leq k \leq p-2$).
- Compute $c1 = g^k \bmod p$.
- Compute $c2 = m * h^k \bmod p$.
- Output the ciphertext pair $(c1, c2)$.

3. Decryption:

- Compute $s = c1^a \bmod p$.
- Compute the modular inverse $s^{-1} \bmod p$ using the Extended Euclidean Algorithm.
- Recover the message $m = c2 * s^{-1} \bmod p$.

4. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Confirm g is a primitive root by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure a , k , and m are within valid ranges and s has a modular inverse.

5. Output:

- Display the public key (p, g, h) , private key a , random k , ciphertext $(c1, c2)$, and decrypted message.
- Verify decryption success if the decrypted message matches the original.

Source Code:

```
#include <stdio.h>
#include <stdbool.h>
long long gcd(long long a, long long b){
    while(b){long long t=b;
        b=a%b; a=t; }
    return a; }
bool is_prime(long long n){
    if(n<=1) return false;
    if(n==2) return true;
    if(n%2==0) return false;
    for(long long i=3; i*i<=n; i+=2)
        if(n%i==0) return false;
    return true;}
long long mod_pow(long long base, long long
exp, long long mod){ long long result=1;
    base%=mod;
    while(exp){
        if(exp&1) result=(result*base)%mod;
        base=(base*base)%mod; exp/=2;}
    return result;}
bool is_primitive_root(long long g, long long p){
    if(!is_prime(p) || gcd(g,p)!=1)
        return false;
    for(long long i=1; i<p-1; i++)
        if(mod_pow(g,i,p)==1) return false;
    return mod_pow(g,p-1,p)==1; }
long long mod_inverse(long long a, long long m){
    long long m0=m, x0=0, x1=1;
    if(m==1) return 0;
    while(a>1){ long long q=a/m;
        long long t=m; m=a%m; a=t;
        t=x0; x0=x1-q*x0; x1=t; }
    if(x1<0) x1+=m0; return x1; }
void elgamal(long long p, long long g, long long
a, long long m, long long k){
    if(!is_prime(p)){printf("Invalid: p must be
prime\n");
        return;}
    if(!is_primitive_root(g,p)){ printf("Invalid: g
must be a primitive root of p\n");
```

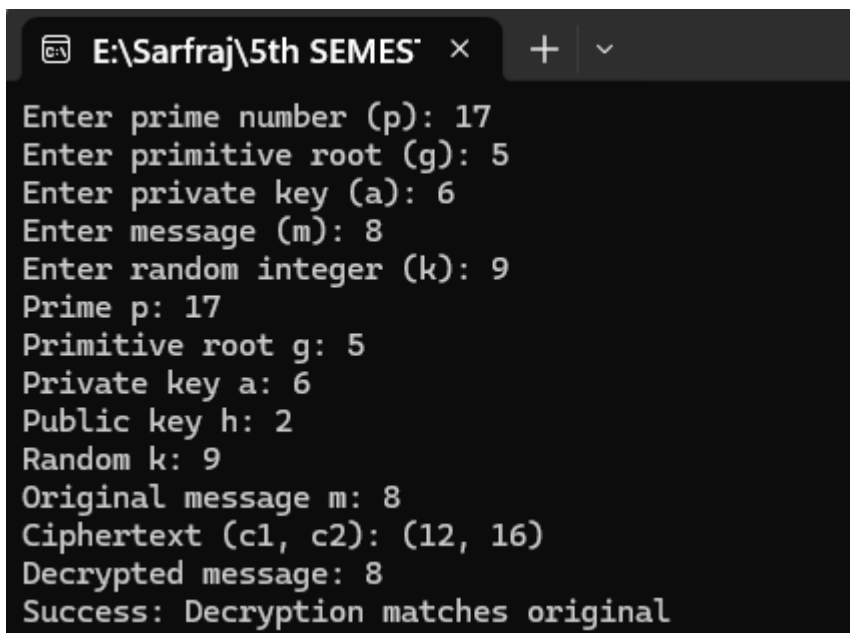
```

        return;}
    if(a<1 || a>p-2){
        printf("Invalid: private key a must be in
range [1,p-2]\n");
        return;}
    if(m<0 || m>=p){
        printf("Invalid: message m must be in range
[0,p)\n");
        return;
    }
    if(k<1 || k>p-2){
        printf("Invalid: random k must be in range
[1,p-2]\n");
        return;
    }
    long long h=mod_pow(g,a,p);
    long long c1=mod_pow(g,k,p);
    long long c2=(m*mod_pow(h,k,p))%p;
    long long s=mod_pow(c1,a,p);
    long long dec=(c2*mod_inverse(s,p))%p;
    printf("Prime p: %lld\n",p);
    printf("Primitive root g: %lld\n",g);
    printf("Private key a: %lld\n",a);
    printf("Public key h: %lld\n",h);
    printf("Random k: %lld\n",k);
    printf("Original message m: %lld\n",m);
    printf("Ciphertext (c1, c2): (%lld,
%lld)\n",c1,c2);
    printf("Decrypted message: %lld\n",dec);
    if(dec==m)
        printf("Success: Decrypted message matches
original\n");
    else
        printf("Error: Decryption failed\n");
}
int main(){
    long long p,g,a,m,k;
    printf("Enter prime number (p): ");
    scanf("%lld",&p);
    printf("Enter primitive root (g): ");
    scanf("%lld",&g);

```

```
    printf("Enter private key (a): ");
    scanf("%lld",&a);
    printf("Enter message to encrypt (m): ");
    scanf("%lld",&m);
    printf("Enter random integer (k): ");
    scanf("%lld",&k);
    elgamal(p,g,a,m,k);
    return 0;
}
```

Output:



```
E:\Sarfraj\5th SEMES' x + v
Enter prime number (p): 17
Enter primitive root (g): 5
Enter private key (a): 6
Enter message (m): 8
Enter random integer (k): 9
Prime p: 17
Primitive root g: 5
Private key a: 6
Public key h: 2
Random k: 9
Original message m: 8
Ciphertext (c1, c2): (12, 16)
Decrypted message: 8
Success: Decryption matches original
```